Fomi Frontend Developer Assessment

- **Candidate:** Muhammad Arsalan Ishaq
- **Live app:** https://fomi-pink.vercel.app
- **Repository:** https://github.com/aslan997/fomi

Deliverable	Link
Part A: Mockup implementation	https://fomi-pink.vercel.app/implementation
Part B: Original workspace (Fomi Studio)	https://fomi-pink.vercel.app/workspace
Product thinking document	Included below (`docs/PRODUCT_THINKING.md`)
Architecture and engineering notes	`docs/ARCHITECTURE.md`

Summary

Part A recreates the supplied image generator design using spacing measured from the mockup and colors sampled from it (`docs/PART_A_SPEC.md`). Every visible control is functional: generation through a mocked API with loading, error/retry and empty states, feed filters, history navigation, a gallery, a full preview with keyboard navigation, save, download, reuse of settings, and a dark theme. Layouts are adapted for mobile, tablet, desktop and large screens.

Part B (Fomi Studio) is a creative continuity workspace. Any image can be **branched** with an explicit intent (Lighting, Framing, Subject or Remix), which keeps part of its look and explores the rest. A compact, keyboard-navigable **branch tree** shows lineage and the active path. **Compare** works across branches and explains the differences through attributes and a word-level prompt diff, and **Keep** preserves the chosen results.

Engineering highlights

- Next.js 16 App Router, JavaScript and Tailwind v4 design tokens, with no UI kit or state management library.
- A mock API with shared validation, deterministic results and simulated failure modes. Part B variations are pre-rendered families of color grades and framings, so each branch is visibly continuous with its parent.
- Native `<dialog>` modals, the ARIA tree pattern, live-region announcements and reduced-motion support.
- 56 Playwright tests on desktop and mobile covering the API, the workflows in both apps and axe accessibility scans. Lint and the production build are clean.

How to review

1. Open **Part A**, type a prompt and press Generate (or Ctrl+Enter). To see the error and retry flow, open **Support > Demo controls** and select **Fail** before generating.
2. Open **Part B**, hover over an image and choose **Branch > Lighting**. Use the **Branches** panel (or the arrow keys) to move through the lineage, then select images from two different branches and click **Compare**.

Product Thinking (Part B)

Concept: a creative continuity workspace. Each result can become the starting point of a branch that keeps what is working and changes one thing at a time, instead of every generation being a separate, unrelated batch.

---

1. What problem were you trying to solve?

User. A professional such as an art director, brand designer or content lead who generates images every day for real projects: a campaign, a product launch, a series. They are not looking for a single good image. They need a consistent set that fits a direction the client has already approved.

Current friction. Most generation tools are built around a prompt box and a results feed. This works well for the first idea, but it becomes difficult by the tenth iteration:

- **Each run starts from scratch.** Re-prompting to change the light usually changes the subject, crop and mood as well, so users spend effort trying to hold on to the parts they liked.
- **History is a timeline rather than a structure.** A long chronological feed does not show which result a later run came from, or why it looks different. Getting back to "the version before we made it warmer" means scrolling and guessing.
- **Good decisions get lost.** The best frame from Tuesday sits somewhere in a feed of 200 images, separated from the reasoning that produced it.

Why continuity matters. Creative work narrows down over time: the subject is settled first, then the light, then the framing. A tool that lets users keep some variables fixed while exploring others, and that records how they got there, fits the way professionals actually iterate. The workspace is designed around that idea.

---

2. The three most important UX decisions

a) Branching from an image with an explicit intent

- **Problem:** "make a variation" is ambiguous, because it is unclear which part should vary. Users compensate by over-describing the prompt to protect what they like.

- **Decision:** every image has a **Branch** action with four intents: Lighting, Framing, Subject and Remix. Each one states what it **keeps** and what it **explores** (for example, "Keeps subject & framing, explores light"). Lighting, Framing and Subject run in one click. Remix opens the composer with the parent's prompt so that edits are deliberate. An optional one-line direction ("colder, dusk") can steer the result.
- **Trade-off:** a fixed set of intents is less expressive than free-form controls such as per-attribute sliders, masks or reference images. I chose four intents that match how art directors describe changes, and accepted that it is not exhaustive.
- **Benefit:** one decision and one click produce a result that is visibly continuous with its parent. In the demo, a Lighting branch from the desert portrait returns the same scene in four color grades rather than four new scenes.

b) A compact branch tree instead of a node graph or a flat feed

- **Problem:** users need to understand where they are and how they got there within a few seconds.
- **Decision:** a narrow tree panel. Each row shows the run label (G4), the intent ("Subject · #2" means it was branched from image 2), the direction note, three thumbnails and the number of kept images. Connectors show parentage, and the **active path is highlighted in amber** from the root to the current run. The canvas header repeats this as a breadcrumb (`G1 › G2 › G4`) and shows the parent thumbnail with what was kept and what was explored. The panel is implemented as an ARIA tree and is fully keyboard-navigable.
- **Trade-off:** a node-graph canvas would show more relationships at once and could support merging, but it takes a lot of screen space away from the images and has a learning curve. The tree can be read at a glance and stays out of the way.
- **Benefit:** returning to "the version before it got warmer" is one click on the parent row or the breadcrumb, and every result keeps a record of where it came from.

c) Selection across branches, with a compare view that explains differences

- **Problem:** the final decision is often between candidates from different directions, for example the golden close-up from one branch and the cold wide shot from another.
- **Decision:** selection is global (up to four images) and persists while navigating the tree. **Compare** shows the selected images side by side with their lineage, highlights the attributes that differ (light, framing, scene, branch type, seed) and **diffs the prompts word by word**. Branch pages show the same prompt diff against their parent. Starring an image **keeps** it; kept images are marked in the tree and collected under Kept assets.
- **Trade-off:** comparison is limited to four images for legibility, and there are no pixel-level tools such as synced zoom or onion skinning.
- **Benefit:** comparisons are explained as well as displayed, which helps when justifying a choice to a client, and preferred versions stay easy to find.

3. Features intentionally excluded

- **Node-based editing and branch merging.** Powerful but heavy. The tree covers the most common needs (where did this come from, and how do I get back) without adding a second canvas.
- **Real AI integration.** The brief asks for a mocked backend. The mock is deterministic and transparent: variations are pre-rendered color grades and crops of the parent's scene, which demonstrates continuity without implying that images are being generated.
- **Collaboration** (comments, shared cursors, approvals). Valuable for agencies, but it is a separate product surface. The data model (a flat, append-only list of generations with parent references) would extend to it.
- **Inpainting, reference images and per-attribute sliders.** I kept the control surface small: prompt, model, aspect ratio and image count, with seed lock and negative prompt under Advanced.
- **Full asset management** (folders, tags, export presets). Kept assets is the minimal version: the decisions made across all branches in one place.
- **Multiple projects.** The demo opens a single seeded project, and the Projects menu states this instead of showing a placeholder list.

4. Which products inspired the design, and where it differs

- **Figma:** selection-driven UI where actions appear in context on the canvas, quiet interface chrome that keeps the work in focus, and a structural side panel. *Difference:* the side panel shows lineage rather than layers, and there is no free-form canvas.
- **Cursor:** AI changes presented as reviewable diffs rather than silent replacements. This led directly to the word-level prompt diff on branch pages and in Compare.
- **Krea:** fast, low-friction iteration where generating again is cheap. *Difference:* each iteration is attached to a parent and an intent instead of being added to a stream.
- **Higgsfield:** preset-driven creative controls and a results feed (the Part A mockup follows this pattern). *Difference:* Part B replaces the feed with a tree and a single focused run.
- **Leonardo.ai:** follow-up actions on individual images, such as creating variations from a result. *Difference:* the follow-up states what is kept, and the relationship is recorded and navigable.

5. What would you improve with another month?

1. **User validation first.** Five to eight sessions with working art directors on a real brief, to check whether the four intents match their mental model and whether "kept" is the right concept or whether they need separate "approved" and "maybe" states.
2. **Persistence and sync.** Move projects from `localStorage` to a backend with optimistic updates. This is required for anything beyond a demo.
3. **Real generation pipeline.** Map intents to model controls (seed and prompt locking, image-to-image strength, ControlNet-style structure locks) and stream partial results.

4. **Richer comparison.** Synced zoom and pan, A/B flicker, and a "promote to hero" action that pins a branch.
5. **Scaling the tree.** Search and filtering (by intent or kept status), automatic collapsing of inactive branches, and "jump to kept". Projects with more than 200 runs will need this.
6. **Collaboration.** Sharing a branch as a review link, with comments on individual images.
7. **Accessibility testing with assistive technology.** The automated axe checks pass; the next step is testing the tree and the compare view with NVDA and VoiceOver users.

---

6. What differentiates the product?

Creative Branching. Most tools are built to show more images. Fomi Studio is built to show a specific image with one deliberate change, such as different light, and to record that the change was requested.

- Any result can become a parent. A branch states what it keeps and what it explores, so continuity is the default rather than something the user has to protect through prompt wording.
- The project is structured as a tree of decisions rather than a feed of outputs. Users can see the path to the current image, return to any earlier version and compare across directions with the differences explained.
- Kept images turn exploration into a record of choices.

Generating another batch gives more options, while continuity helps a project converge on fewer, more consistent results with a clear explanation of how they were reached. For professionals working to a brief, that is usually the harder part, and it is what this workspace focuses on.